ML2++: A Model-Driven Blended Framework for Automated Machine Learning, Time Series Forecasting, and Data Visualization in IoT Applications

Zahra Mardani Korani
CICTI, Laboratório Nacional de Engenharia Civil (LNEC)
ISTAR, Instituto Universitário de Lisboa (ISCTE-IUL)
Lisbon, Portugal
zmardani@lnec.pt

Thimoty Smet, Moharram Challenger
Dept. of Computer Science
University of Antwerp & Flanders Make
Antwerp, Belgium
thimoty.Smet@student.uantwerpen.be
moharram.challenger@uantwerpen.be

Armin Moin
Dept. of Computer Science
University of Colorado Colorado Springs
Colorado Springs, USA
amoin@uccs.edu

Joao Carlos Ferreira
ISTAR, Instituto Universitário de Lisboa (ISCTE-IUL)
Faculty of Logistics, Molde University College
Lisbon, Portugal; Molde, Norway
joam@himolde.no

Alberto Rodrigues da Silva
INESC-ID, Instituto Superior Técnico, Universidade de Lisboa
Lisbon, Portugal
alberto.silva@tecnico.ulisboa.pt

Gonçalo Vitorino Jesus
CICTI, Laboratório Nacional de Engenharia Civil (LNEC)
Lisbon, Portugal
gjesus@lnec.pt

***Abstract*—Intelligent Internet of Things applications must seamlessly combine established software engineering practices with various machine learning (ML) and time series forecasting techniques. However, most pipelines today rely on handwritten code, which is error-prone and imposes a steep barrier for domain specialists. Domain-specific modeling languages (DSMLs), the backbone of model-driven engineering (MDE), address this complexity by allowing developers to describe systems at a high level, while tooling generates the executable code. To this end, we introduce the ML2 family of DSML frameworks. ML2 converts concise Textula models into complete Python pipelines for classification and regression ML models, autogenerating every preprocessing, training, and prediction script. Building on that foundation, ML2+ adds declarative, end-to-end support for time series forecasting: users can select any statistical (e.g., ARIMA or ETS), machine learning (e.g., SVR or XGBoost), deep learning (e.g., LSTM or Transformer), or hybrid (e.g., Prophet) algorithms and specify key settings, such as lag length or seasonality, without writing a single line of code. In addition, ML2++ enhances both textual and graphical workflows by providing an improved Textula editor alongside Sirius Web, an IDE drag-and-drop graphical modeling environment, and real-time dashboards; ML2++ automatically generates all relevant plots, including loss curves, autocorrelation functions, and forecast versus actual charts during preprocessing, training, and prediction for both ML models and time series forecasters. We validated the framework on two real-world cases, river flow forecasting and server crash prediction, showing faster development, fewer errors, and interpretable results compared to hand-coded workflows.**

***Index Terms*—Model-Driven Engineering, Machine Learning, IoT, Time Series Forecasting, Visualization, ML2 Framework**

I. Introduction

The rapid growth of the Internet of Things (IoT) has resulted in a substantial increase in both the volume and velocity of sensor-generated data, especially time-series data [1]. In various IoT-driven applications, such as predictive maintenance, anomaly detection, environmental monitoring, and real-time optimization, accurate prediction of temporal data is not only essential but often critical to mission success. However, traditional methods for developing time-series models are largely manual, necessitating precise setups, extensive domain expertise, and repetitive debugging. Consequently, the development process becomes protracted, prone to errors, and frequently inaccessible to non-expert users [2], [3]. Recent improvements in Model-Driven Engineering for the Internet of Things (MDE4IoT) have shown potential in solving these problems by focusing on creating high-level models instead of getting into the details of implementation. A study by Naveed et al. [4] found that MDE4ML methods can automatically turn high-level software models into code and manage the training, deployment, and maintenance of ML components, which simplifies the process and speeds up development. MDE4IoT frameworks provide automated code generation and

behavior modeling, allowing developers to focus on building modular and reusable models rather than writing complex code [5]. This paradigm greatly improves the development, consistency, and maintainability of intelligent and data-driven IoT systems. The ML2+ framework, which builds on ML2 [6], greatly improves the automation of tasks for predicting IoT time series data [7]. By using a model-driven approach, ML2+ automated key tasks like preparing data, training time-series models, and evaluating them, which made model development more consistent and less reliant on manual work. Moreover, ML2+ extends ML2 and also supports conventional ML models for classification and regression, allowing users to specify both static and temporal analytics within the same DSML. However, ML2+ relied entirely on a textual domain-specific modeling language (DSML), which, despite its flexibility, offered limited usability. Specifically, it did not include real-time visualization or interactive diagnostics, making it difficult for users to observe model behavior, trace errors, or understand results without extensive manual inspection. These limitations are particularly critical in IoT environments, where rapid model iteration, transparency, and interpretability are paramount. Both developers and domain experts benefit from quick visual feedback, tracking errors in real time, observing prediction patterns, and analyzing residuals to detect problems and improve models without delay. To overcome these barriers, this paper proposes ML2++, a comprehensive enhancement of the ML2+ framework. ML2++ automates code generation and execution of key stages (preprocessing, training, prediction, visualization); model-specific parameters remain user-configurable for time series forecasting while introducing several novel capabilities. Most importantly, it includes an advanced visualization module that generates diagnostic and performance plots using widely adopted libraries such as Matplotlib and Plotly. Model specifications automatically create these visual outputs, which provide interactive real-time information on how well the time series model is performing, how training is going, and the relationships over time. This functionality significantly simplifies the process of solving issues and adjusting settings. ML2++ introduces a dual editing environment that includes both a robust textual editor and a graphical editor developed with Eclipse Sirius Web. This dual interface caters to a diverse user base: the textual editor provides fine-grained control, while the graphical editor enables intuitive drag-and-drop visual modeling, making it accessible to non-programmers and domain specialists. By facilitating seamless transitions between these two modes, ML2++ enhances usability, minimizes configuration errors, and accelerates the deployment pipeline. The remainder of this paper is organized as follows. Section II reviews related work on MDE4IoT and automated machine learning frameworks. Section III introduces the extended domain models and the overall architecture of ML2++. Section IV details the metamodel and the dual-editing infrastructure. Section V describes the functionalities of both the textual (Textula) and graphical (Sirius-Web) editors. Section VI demonstrates the capabilities of the framework through real-world use cases. Finally, Section VII concludes the paper and discusses the directions for future development.

II. Related Work

Model-driven engineering (MDE) has increasingly been combined with machine learning (ML) to tackle the complexities of IoT systems, where automation, scalability, and interpretability are paramount [8]. ML2 [9] pioneered this integration by providing a domain-specific textual modeling language to generate code for classical classification and regression tasks. Its successor, ML2+ [7], added built-in support for time series forecasting, e.g., models such as LSTM, ARIMA and Prophet, and automated data preprocessing, training, and evaluation. Despite these advances, ML2+ remained a textual tool, limiting interactive analysis and real-time visualization. Various GUI-based and code-centric platforms occupy the lower end of the MDE spectrum (as summarized in I). Libraries such as TensorFlow/Keras [10], [11] offer unmatched deep learning flexibility but place the entire pipeline in the user's hands, requiring custom scripts for data wrangling, model orchestration, and deployment. Visual flow environments such as KNIME [12] and RapidMiner [13] reduce boilerplate with drag-and-drop workflows, yet they emit proprietary code fragments and rely on extensions for forecasting. The interchange standards PMML [14], ONNX [15], and PFA [16] focus on model portability but do not execute pipelines or produce diagnostics. Probabilistic toolkits such as Infer.NET [17] provide Bayesian inference blocks in C# but still demand manual wiring of data ingestion, model fitting, and result reporting. Together, these tools illustrate a trade-off: high algorithmic expressiveness at the cost of automation and end-to-end reproducibility. At the other extreme, pure MDE platforms and cloud templates emphasize metamodel-driven code generation, but omit analytics. ThingML/HEADS [18], [19] and µ-Kevoree [20] deliver high levels of automation and runtime adaptability for IoT behaviors, yet leave forecasting and performance plots to external frameworks. Similarly, Stratum [21] and SysML-based CPS design tools [22] streamline infrastructure wiring but do not generate training or visualization code. Textual AutoML DSLs such as MDE-Forecast [23] simplify the time series configuration with ARIMA and Prophet blocks but delegate feature engineering and plotting to the user, limiting immediate interpretability. None of the approaches surveyed combines fully declarative DSML, dual editing modes, and embedded real-time visualization. ML2++ framework addresses this gap: it unifies the Textula textual DSL with a Sirius-Web graphical canvas, automatically generates end-to-end Python pipelines (preprocessing, training, forecasting), and injects Matplotlib/Plotly plots directly from the model specification. Thus, ML2++ delivers full automation, comprehensive time-series (TS) support, strong MDE alignment, and high interpretability in a single cohesive environment.

III. Proposed Approach

ML2++ extends the earlier ML2 [9] and ML2+ [7] frameworks by (i) attaching explicit *visual-analytics* attributes to every domain model and (ii) offering both textual (Textula)

TABLE I: Comparison of related frameworks for ML integration in MDE4IoT.

Framework	Interaction Type	Automation Level	ML Support	Time-Series Support	MDE Alignment	Interpretability	Code Generation
TensorFlow/Keras [10], [11]	Code (Python)	Manual	Deep Learning	No (Custom Only)	No	High (with effort)	No
KNIME/RapidMiner [12], [13]	GUI (Visual Flow)	Partial	Classical + DL	Limited (Plug-ins)	Partial	Medium	Partial (Limited Export)
PMML / ONNX / PFA [14], [15]	Interchange Format	None (Post-Training)	Model Export Only	No	N/A	Low	Format-Only
Infer.NET [17]	Code (C#)	Partial	Probabilistic ML	No	No	Medium	Limited (C# Only)
ThingML / HEADS [18], [19]	Text + GUI (MDE)	High	N/A	No	Yes	N/A	Yes (IoT DSLs)
µ-Kevoree [20]	Models@Runtime	High	N/A	No	Yes	N/A	Runtime Models
GreyCat [24]	Code + Model DSL	Medium	Predictive Models	Partial	Yes	Medium	No
MoSIoT [25]	GUI (Scenario Modeling)	Medium	Scenario-Level ML	No	Yes	Low	No
Stratum [21]	Cloud Platform GUI	Medium	Analytics Pipelines	No	Partial	Medium	No
Model-based Design for CPS [22]	DSL + SysML	Partial	ML-in-the-loop	No	Yes	Medium	Not Specified
MDE-Forecast [23]	Textual Editor	Partial	AutoML (External Only)	Yes (AutoML TS Configs)	Yes	Low	Config Only
ML2 (ML-Quadrat) [9], [26], [27]	Textual DSL	Partial	Supervised ML, semi-supervised ML, unsupervised ML	No	Yes	Medium	Yes (Non-TS)
ML2+ [7]	Textual DSL	High	Supervised ML, semi-supervised ML, unsupervised ML, Time-Series models	Yes	Yes	Medium	Yes
ML2++	Hybrid (Text + GUI)	Full	Supervised ML, semi-supervised ML, unsupervised ML, Time-Series models	Yes (Built-in)	Yes	High (via plots)	Yes (All Scripts)

and graphical (Sirius-Web) editors. We group the resulting contributions into three parts:

1) **Extended Standard-ML Domain Model**
2) **Extended Time-Series Domain Model**
3) **Smart Software Model (SSM)** that unifies 1) and 2)

The paragraphs below explain the new metamodel elements, code-generation rules, and automatic plot injection introduced by ML2++.

1) Extended Standard-ML Domain Model: Building on the ML2 schema of Moin *et al.* [9], we add a visual-analytics component V_{ML}:

$$DM'_{\text{ML}} = \Big\{ ml \;\Big|\; ml = \big(\upsilon_{ML}, P_{ML}, \Phi_{ML}, H_{ML}, I_{ML}, V_{ML}(ml)\big) \Big\} \tag{1}$$

where

- υ_{ML} — model architecture (e.g., *Logistic Regression*, *Decision Tree*);
- P_{ML} — model parameters (e.g., regularisation strength, number of neurons);
- Φ_{ML} — input-feature set;
- H_{ML} — training hyper-parameters (e.g., learning rate, batch size);
- I_{ML} — metadata (e.g., author, version);
- $V_{ML}(ml)$ — **new in ML2++**; holds visual-analytics attributes (e.g., prediction-vs-actual, learning curves) that trigger automatic plot generation.

2) Extended Time-Series Domain Model: Starting from the time-series specification introduced in ML2+ [7], ML2++ augments it with the analogous visual-analytics attribute V_{TS}:

$$DM'_{\text{TS}} = \Big\{ ts \;\Big|\; ts = \big(\upsilon_{TS}, P_{TS}, \Phi_{TS}, H_{TS}, I_{TS}, V_{TS}(ts)\big) \Big\}, \tag{2}$$

where

- υ_{TS} — time-series architecture (e.g., *LSTM*, *ARIMA*, *Prophet*);
- P_{TS} — model parameters (e.g., lag length, forecast steps, seasonality);
- Φ_{TS} — engineered time-series features;
- H_{TS} — training hyper-parameters;
- I_{TS} — metadata (e.g., forecast horizon);
- $V_{TS}(ts)$ — **new in ML2++**; governs forecast-vs-actual plots, overfitting diagnostics, etc., automatically injected into the generated scripts.

3) Smart Software Model (SSM): Following the integration strategy of the earlier frameworks [7], [9], ML2++ combines the two extended domain models as

$$SSM = \big(A,\ \Psi,\ f_B\big(DM'_{\text{ML}}, DM'_{\text{TS}}\big),\ C\big) \tag{3}$$

and thus propagates both analytics logic and visualisation directives to code-generation time.
where

- A — annotations (e.g., external library references);
- Ψ — structural elements (e.g., *Things*, *Ports*, *Messages*);

- C — configuration settings;
- f_B — behaviour-integration function that fuses DM'_{ML} and DM'_{TS} into executable runtime behaviour and embeds the corresponding visualisations automatically.

IV. Architecture of ML2++

ML2 ++ extends the ML2 + framework by adding new features to the Data Analytics Modeling Language (DAML), including built-in visualization tools in the metamodel and Xtext grammar, as well as two ways to edit (text and graphics). This integration supports a seamless and systematic management of visualizations, significantly improving the interpretability and analytical clarity of the model compared to the previous ML2+ framework. The architecture of ML2++ features clearly defined visualization attributes within its metamodel. These attributes categorize visualization features into various analytical steps, including data preparation, prediction verification, future forecasting, and overfitting analysis. By including these visualization features directly in the grammar, ML2++ allows users to clearly define and control how visualizations behave using attribute-based expressions, which is an improvement over the keyword method used in ML2+. Table II provides a detailed overview of the visualization attributes categorized by their analytical functions, emphasizing their organized integration into the ML2++ framework. This structured methodology promotes flexibility, clarity, and user-friendliness in handling visualization tasks within data analytics workflows.

TABLE II: Visualization attributes in ML2++ categorized by analytical function.

Analytical Category	**Visualization Attributes**
Pre-processing	LinePlot, Histogram, BoxPlot, ScatterPlot, HeatMap, AutoCorrelationPlot, LagPlot, ViolinPlot, CorrelationMatrixPlot, PairPlot
Prediction Diagnostics	PredictionVsActualPlot, ResidualsPlot, ErrorDistributionPlot, ConfidenceIntervalsPlot, ConfusionMatrixPlot, RocCurvePlot, PrecisionRecallCurvePlot
Forecast Horizon	ForecastVsActualPlot, ForecastErrorPlot, ForecastIntervalsPlot
Overfitting Analysis	TrainingLossPlot, ValidationLossPlot, LearningCurvePlot, BiasVarianceTradeoffPlot

V. Blended Modelling Environments

ML2++ offers two complementary user interfaces (Figure 1 and Figure 2) (i) the Textula textual editor and (ii) the Sirius graphical editor for drag-and-drop modeling.

A. Textual Editor: Textula

Textula is the domain-specific textual environment first released with ML2+ [7] and enhanced in ML2++. It provides a rich language workbench that features syntax highlighting, autocompletion, live metamodel validation, and snippet libraries built on a concise DSL that supports deep learning, statistical, and classical ML forecasters. Using this DSL, users can clearly outline each step of an IoT time series pipeline, which includes choosing data, preparing features (e.g., creating lag windows, resampling, filling in missing values, and removing outliers), adjusting settings for the model, and defining how to evaluate results, such as prediction outcomes and performance measures (e.g., MSE and accuracy), as well as detecting outliers, grouping data, and adding context notes. This language allows for a comprehensive configuration of the entire data analytics workflow, from data ingestion to preprocessing, modeling, visualization, and evaluation, enabling full automation and streamlined IoT time series forecasting. Visualizations can be specified within the model itself, by including plot configurations within the `model...` section (see Listing 2) or by inserting a `visualization...` block at the bottom of the Data Analytics section (see Listing **??**) to request preprocessing plots without writing Python. Within Textula, six buttons streamline the workflow. The first button saves the model as a `.thingml` file, which parses, validates, and persists the EMF graph. The second button executes the preprocessing script and immediately displays the resulting plots inline. The third button invokes the Eclipse-created Xtend-based code generator, packaged as the Java action compiler. `mlquadrat.compilers.registry-2.0.0-SNAPSHOT-jar-with-dependencies.jar` to produce Python scripts for preprocessing, training, prediction, and visualization. The fourth button downloads all generated scripts as a ZIP archive. The fifth button compiles and runs either the Java artifacts or the Python scripts in a sandbox, streaming console output to the IDE. The sixth button runs the training and prediction scripts to render forecasting plots such as loss curves and ROC charts inline, providing immediate visual feedback. Additionally, users can access the same Xtext-based editor and code generation features through a browser at https://ml2plusplus.jvmhost.net/, eliminating the need to install Eclipse locally. A significant enhancement in ML2++ is the embedded visualization layer, where any plot type listed in the DSL is automatically converted into executable code. For example:

Listing 1: Per-model plot configuration

```
visualization {
  ...
plots LINE_PLOT
}
```

The `visualization` block configures preprocessing plots, such as line and histogram charts, which are generated and displayed immediately when the preprocessing script is run. In contrast, plots specified inside the `model{...}` section focus on training and forecasting diagnostics (e.g., overfitting and forecasting plots):

Listing 2: Per-model plot configuration

```
model {
  ...
  overfitting_plots TRAINING_LOSS,
  forecasting_plots FORECAST_ERROR
}
```

Including such plot settings causes ML2++ to inject the appropriate Matplotlib logic into `DA_Preprocess.py`, `DA_Train.py`, and `DA_Predict.py`. Running these scripts or clicking the plot buttons produces all requested visualizations from preprocessing summaries to training and forecasting diagnostics without manual coding.

B. Graphical Editor: Sirius-Web

Sirius Web [28] is a graphical studio that transforms a metamodel into multiple interactive diagrams, ranging from standard

Fig. 1: ML2++ Textula quick-start DSL editor.

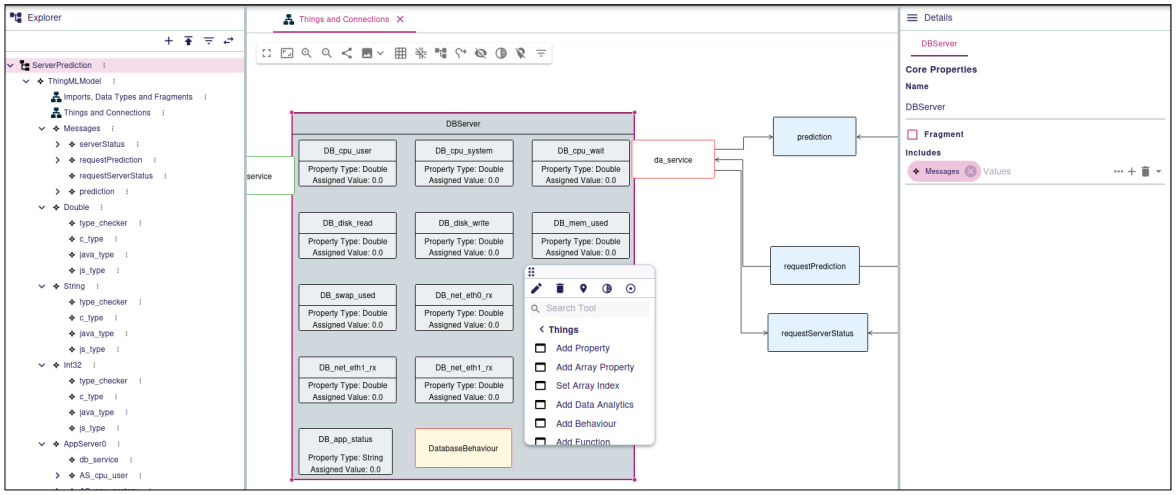


Fig. 2: ML2++ Sirius-Web graphical editor.

form-based interfaces to drag-and-drop visualizations. The multiple viewpoints of ML2++ in Sirius-Web are implemented to reflect the typical structure of a ThingML file created in a text editor. The sources for the graphical editor are available on GitHub [29]. A standard ThingML file is organized into three main sections. The first section includes imports, fragments, and datatype definitions, which are declared at the beginning and referenced throughout the model. The second section defines the ”things” that represent communicating entities, using the previously defined fragments and datatypes. The final section, configuration, connects and configures these things to form an executable model. If the ML2++ diagrams are not preloaded in Sirius-Web, users can manually download and import them into the editor. To begin, the user should create a new project and a model based on the ThingML grammar. From the model's root element, two main graphical views can be created: “Imports, Fragments, Messages” and “Things Connections.” To create a view, the user selects the root element in the tree structure, right-clicks, and chooses a representation. It is recommended to start with the “Imports, Fragments, Messages” view, which sets up the foundational elements used in later stages. When a view is created, an empty canvas is shown. Right-clicking on the canvas opens a context menu whose options depend on the selected element. In the “Imports, Fragments, Messages” view, users can create imports, datatypes, and fragments. Right-clicking on a fragment provides additional actions such as creating messages associated with that fragment. Newly created elements appear on the canvas with their graphical icons, and their attributes can be edited in the property panel on the right. After defining the datatypes and fragments, the user creates the second view, “Things Connections.” This view is more complex and supports defining things and their internal elements. Each thing may include ports (which use predefined messages for communication), variables, user-defined functions, behavior modeled through statecharts, and data analytics components. The behavior of

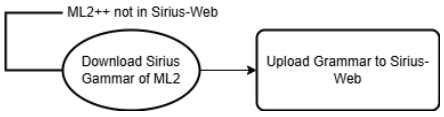


Fig. 3: ML2++ web-based modelling workflow: Adding the Sirius-Web grammar to the editor

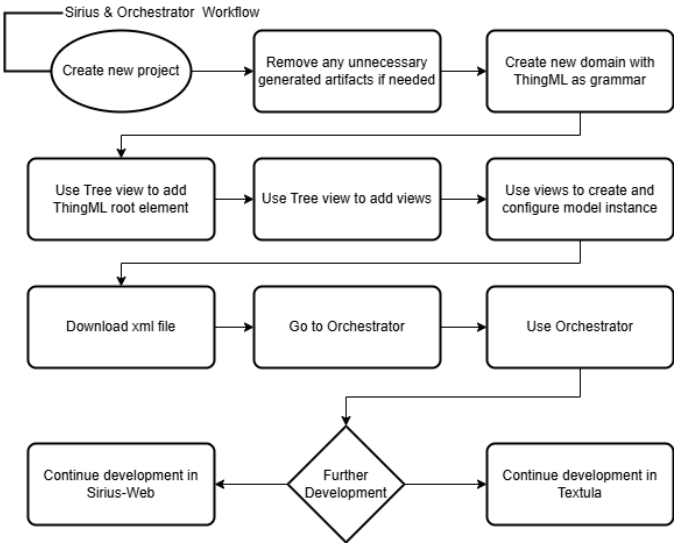


Fig. 4: ML2++ web-based modelling workflow: Integration of Sirius-Web for visual model editing

each thing is represented as a statechart diagram showing states and transitions. The Data Analytics viewpoint provides a form-based interface organized into several pages, corresponding to different model elements in ML2++. Toward the end, this interface presents buttons for adding DAML models such as time-series forecasting or standard classification. When a button is clicked, the corresponding model is instantiated and connected to the analytics section. Each DAML model opens its own form where users can edit the parameters specific to that model type. Once all things are defined, users return to the "Imports, Fragments, Messages" view to create a configuration element. This final view links all defined things, specifying how they are instantiated and connected. When the configuration is complete, the user can download the model as an XML file and proceed to the Sirius Orchestrator, a web platform that transforms Sirius-based models into executable ThingML code. The orchestrator allows users to upload their XML file and download each step of the transformation, including the final ZIP archive containing the full `.thingml` project. If the user prefers to continue editing the model textually, the generated `.thingml` file can be imported into the Textula editor for further development. The overall process, from graphical modeling to code generation and textual refinement, is illustrated in Figures 3, 4, 5.

VI. Evaluation

ML2++ has been validated using two IoT use cases: river flow forecasting and Server Crash Prediction. These use cases were chosen to highlight the ability of the framework to

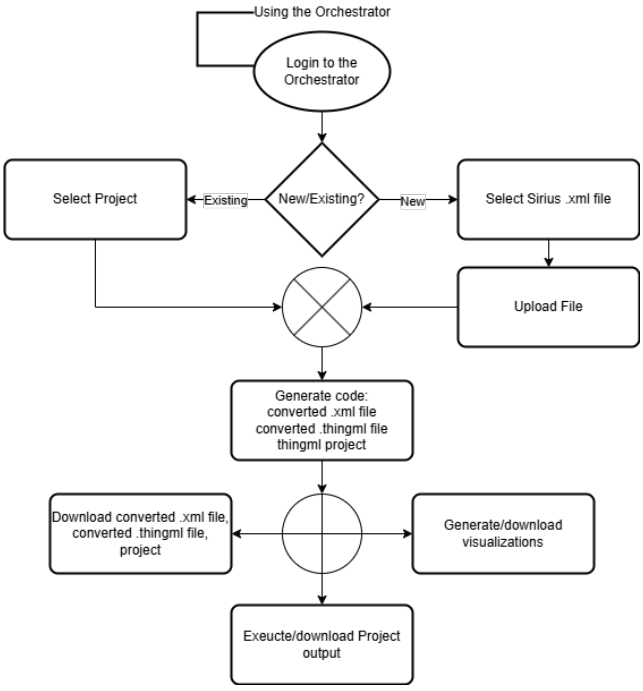


Fig. 5: ML2++ web-based modelling workflow: Integration of the Sirius Orchestrator for visual code generation.

address time series regression tasks that require complex temporal modeling and classification problems typical of industrial monitoring. By comparing these two cases, one from environmental monitoring and the other from IT operations, we illustrate how ML2++ generalizes across diverse datasets, supports a wide range of machine learning models, and automates code/execution for preprocessing, training, prediction, visualization; parameters remain user-configurable. Although ML2++ was evaluated against hand-coded workflows to highlight time and effort savings, a direct comparison with other model-driven or AutoML tools was not performed due to significant differences in their focus and level of integration. We recognize this as a limitation and plan to include comparative benchmarks against similar frameworks in future work.

Table III summarizes the essential characteristics of each use case, highlighting their differences in data types, model selection, and analytics workflows.

TABLE III: Comparison of ML2++ Use Cases

Aspect	River Flow Forecasting	Server Crash Prediction
Domain	Environmental monitoring	IT operations
Target Variable	River discharge (continuous)	Server status (ON/OFF)
Use Case Type	Time-series regression	Binary classification
Dataset	Hydrological time series	IT telemetry metrics
Model	LSTM (deep learning)	Logistic regression (classical ML)

A. River Flow Forecasting

A river flow forecasting case study was previously implemented and presented in our earlier work on the ML2+ framework [7]. In this paper, we build on that case study to demonstrate the extended capabilities of ML2++, specifically its automated visualization support and enhanced DSML workflow integration. The original use case focused on predicting river

discharge at the target station Açude Ponte de Coimbra (12G/01AE) using real-world multivariate datasets. Daily river flow measurements were collected from multiple upstream monitoring stations, including Albufeira da Aguieira (11H/01A), Albufeira da Raiva (12H/01A), Albufeira de Fronhas (12I/01A), and Açude Ponte de Coimbra itself. The dataset spans from January 1, 1984, to November 18, 2024, and contains daily average effluent flow values (in m^3/s) provided by the Portuguese National Water Resources Information System (SNIRH) [30] [31]. Each station transmits its data daily to a central server, which is accessed by the DAML server. The workflow, orchestrated by the DAML components in ML2++, automates code generation and execution of key stages (preprocessing, training, prediction, visualization); model-specific parameters remain user configurable of the forecasting pipeline. The preprocessing phase, handled by `DA_Preprocess`, prepares the multivariate time series data by interpolating missing values (up to 10 consecutive gaps), resampling based on a common period threshold, and converting the dataset into a supervised learning format using lagged features. These steps, implemented via the `Scikit-Learn` library, are essential for capturing temporal dependencies between stations. The resulting dataset is divided into 80% for training and 20% for testing, while preserving temporal order. Training is performed using the `DA_Train` component, which applies an `LSTM` model configured with 50 neurons, 50 training epochs, a batch size of 16, L2 regularization (0.01), dropout (0.2), and the ReLU activation function. The model is trained using the Keras library, with Mean Squared Error (MSE) as the loss function. Following training, the `DA_Predict` component generates forecasts for the next three days at the target station. These values are compared with a flood threshold of $150\,m^3/s$; if exceeded, a flood alert is automatically triggered, supporting proactive decision-making in flood risk management. In ML2++, this same case study is re-evaluated to demonstrate its new visualization capabilities. After executing the DSML model, the system automatically generates Python scripts (`preprocess.py`, `train.py`, and `predict.py`), as well as serialized data in the `pickles/` directory and visual outputs in the `plots/` folder. see Listing 3 presents the instance of the ML2++ model used in this experiment, including the configuration for training and visualization. Figure 6 shows the folder containing the generated plot images, while Figure 7 displays examples of the actual visualizations produced. These outputs include time-series trend plots, common period alignment diagnostics, overfitting curves (e.g., `TRAINING_LOSS`), and forecast-versus-actual comparisons (e.g., `FORECAST_VS_ACTUAL`). The plots are configured using the ML2++ `visualization` block, where users specify plot types such as `LINE_PLOT` and others (see Listing 3). These visualization features improve transparency, support real-time model evaluation, and can be integrated into dashboards or IoT services, thus demonstrating seamless end-to-end automation in the forecasting pipeline.

Listing 3: Data Analytics section of the ML2++ model instance used for the river flow forecasting case study.

```
data_analytics da1 @dalib "keras-tf" {
 data {
  dataset "/home/zahra/flow.csv"
  features daily discharge1, discharge2, discharge3
  output_features discharge2
  timestamps ON
  common_period_threshold 10
 }
 preprocessing { resample DAILY }
 time_series { sequential TRUE steps 3 lag 20 multivariate
     ON supervised_learning ON }
 model {
  Automl OFF
  algorithm LSTM mylstm(
   hidden_layer_sizes (90,90),
   input_activation relu,
   hidden_activation relu,
   output_activation relu,
   regularization L2,
   dropout 0.2,
   optimizer adam,
   rate 0.001,
   batch_size 32,
   epochs 100,
   metrics MSE,
   early_stopping OFF,
   overfitting_Plots TRAINING_LOSS,
   forecasting_plots FORECAST_VS_ACTUAL
  )
 }
 evaluation { prediction_results p1, p2, p3 metrics MSE }
 visualization { plots (LINE_PLOT) }
}
```

Fig. 6: Generated plot files inside the `plots/` directory after running the ML2++ pipeline.

B. *Server Crash Prediction*

ML2++ was newly evaluated on a real-world server crash prediction use case using a private industrial dataset of operational telemetry data from Yurtiçi Kargo's server fleet. The telemetry data included key performance metrics such as CPU usage, disk I/O, memory consumption, and network throughput. These variables were recorded periodically to monitor server health and assess risk of crash. The ML2++ pipeline proceeds as follows. The original data set contained significant noise and inconsistencies, some of which could not be automatically resolved through default preprocessing in ML2++. Therefore, we manually removed unneeded features, particularly those

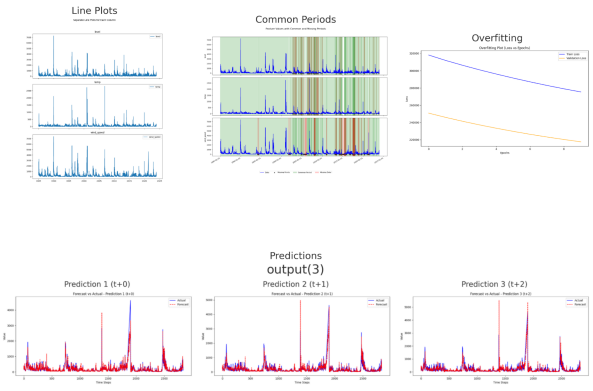


Fig. 7: Visual outputs: time series, overfitting loss, common-period alignment, and forecast-vs-actual plots.

with constant values or no relevance to the binary classification task (predicting server `ON` vs. `OFF` states). The missing values were then addressed using median imputation, which provided robustness against outliers. This cleaned dataset was ingested by the `DA_Preprocess` component, which label-encoded the server status (target variable), normalized all numeric features using `StandardScaler`, and prepared the dataset for modeling. An 80/20 split was applied to divide the data into training and test sets, ensuring that the temporal ordering of the telemetry records was preserved. Training was performed using the `DA_Train` component, which fit a logistic regression model using Scikit-learn. The configuration included L2 regularization, a tolerance value of 0.0001, inverse regularization strength $C = 1.0$, the `lbfgs` solver, and settings for `fit_intercept=True` and `intercept_scaling=1`. This model was chosen because of its simplicity, speed and strong interpretability in binary classification tasks, making it well suited for operational deployment in real-time systems. Training logs and model weights were saved for reproducibility and traceability. Once the model was trained, `DA_Predict` applied the logistic regressor to incoming telemetry streams. The prediction output indicated whether each server was expected to remain `ON` or crash (classified as `OFF`) based on recent behavioral patterns. If a server was predicted to crash, the system automatically issued an alert to the operations team for proactive intervention. Figures 8 and 9 illustrate the ML2++ workflow using the Sirius-Web modeling interface. The former shows the graphical selection and configuration of the Data Analytics model, while the latter depicts the runtime execution flow integrating `DA_Preprocess`, `DA_Train`, and `DA_Predict`. Some plots and metrics will be generated. In this case, from our preprocessing section, a heatmap will be generated showing the relationship between features. For model evaluation, the confusion matrix and the ROC curve were generated by selecting these two variables in the DAML evaluation section.

C. Comparative Analysis

Across both use cases, ML2++ shows substantial improvements in productivity and model quality compared to manual implementations. The current evaluation involved a Ph.D. and a Master's student, both with computer science backgrounds and moderate Python skills. They were familiar with the Sirius-Web and Textula editors. While the results show strong gains in development time and code reduction, we acknowledge the need for broader testing with users from diverse and less technical backgrounds. A larger-scale user study is planned to address this. As summarized in Table IV, in the river flow forecasting scenario, both manual and ML2++ pipelines yielded identical **Root Mean Square Errors (RMSEs)** of 136,m^3/s at $t+0$, 170,m^3/s at $t+1$, and 187,m^3/s at $t+2$ confirming that automation does not compromise accuracy. The Root Mean Square Error (RMSE) [32] used throughout this paper is computed as:

$$\text{RMSE} = \sqrt{\frac{1}{n}\sum_{i=1}^{n}\left(y_i - \hat{y}_i\right)^2} \tag{4}$$

In terms of productivity, ML2++ reduced the development time from 14 h (manual) to 3.5 h, although it requires a 224-line DSML model instance. In contrast, the equivalent manually written Python code totaled just 87 lines. This reflects the shift of boilerplate into the DSML template. Users write a larger, structured model description once, and ML2++ automatically generates all the necessary Python scripts, rather than hand-coding each preprocessing, training, and plotting step. Despite the longer Textula model, its declarative layout means that users focus on what they want: data sources, features, algorithms, and visualizations, while ML2++ handles how it is implemented, reducing mental overhead and error-prone boilerplate. The Server Crash Prediction benchmark further highlights ML2++'s graphical Sirius-Web editor and its seamless integration into the modelling workflow. Here, the primary metric was **classification accuracy (ACC)**; The Classification Accuracy (ACC) [33] used throughout this paper is computed as:

$$\text{accuracy}(y, \hat{y}) = \frac{1}{n_{\text{samples}}} \sum_{i=0}^{n_{\text{samples}}-1} \mathbf{1}(\hat{y}_i = y_i) \tag{5}$$

Both manual and ML2++ pipelines achieved an identical accuracy of 77.9%, while development effort dropped from 5 h (manual) to 2 h (ML2++). The visual editor inevitably produces more lines of `.thingml` code than a hand-crafted textual model, 219 and 212 Lines of code (LOC) respectively, yet the additional verbosity is largely aesthetic, introduced by the code generator to preserve flexibility, rather than functional. Consequently, even with longer generated .thingml files, developer efficiency and end-to-end functionality are markedly higher when using ML2++ because the overall LOC written by the manual users is 502 compared to 219 for the visual editor users.

TABLE IV: Comparison of manual vs. ML2++ implementations for development time, code size, and predictive performance.

Use Case	**Time (h)**		**LOC**		**Metric**[†]	
	Manual	ML2++	Manual	ML2++	Manual	ML2++
River-flow forecasting	14	3.5	224	87	136/170/187	136/170/187
Server-crash prediction.	5	2	502	219	77.9%	77.9%

[†] Regression: RMSE (t+0/t+1/t+2); classification: accuracy.

VII. Conclusion and Future Work

ML2++ enhances ML2+ by introducing an automated visual analytics framework that generates a comprehensive array of graphs for various machine learning models, including classification, regression, and clustering, as well as time series forecasters (e.g., loss and learning curves, ROC and precision recall, autocorrelation, forecast versus actual graphs, and more). Coupled with dual editing capabilities offering both precise

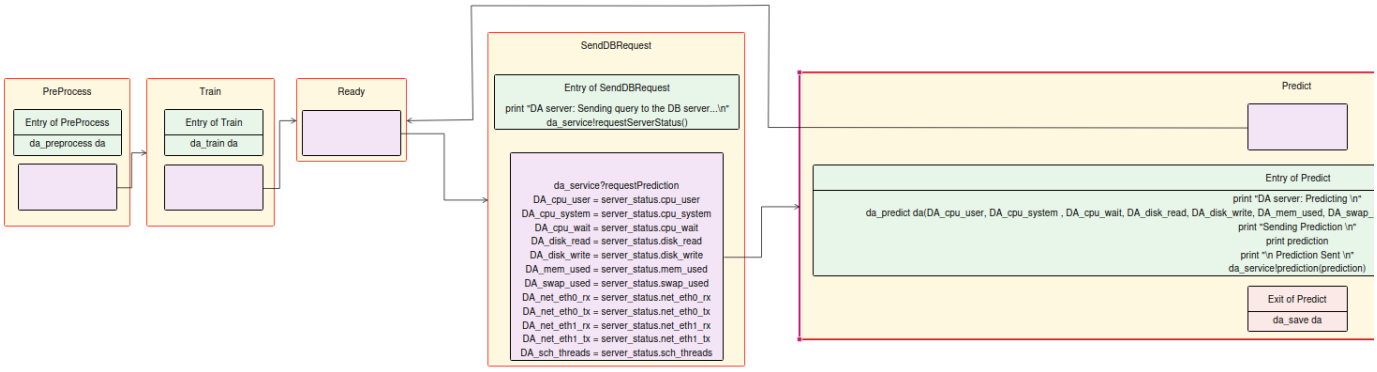


Fig. 8: Statechart view of the Server crash usecase

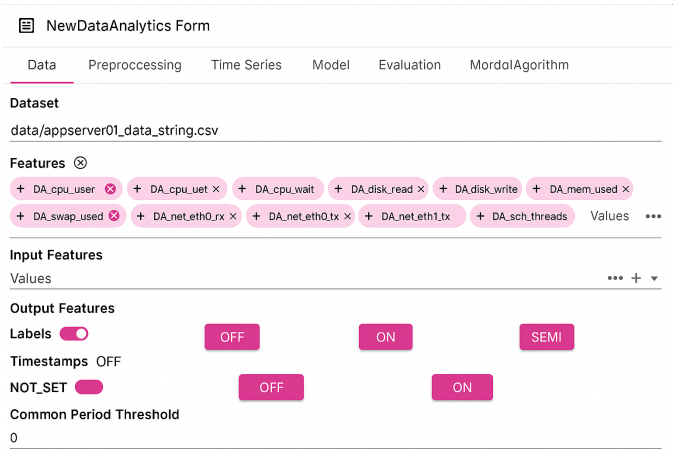
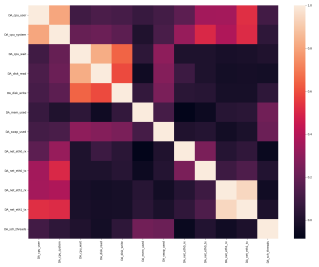
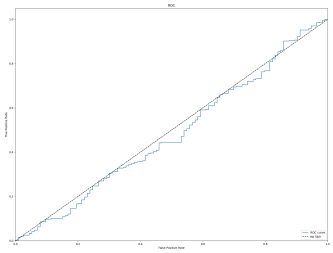


Fig. 9: A section of the ML2++ Sirius-Web editor showing tabs for server crash prediction setup.

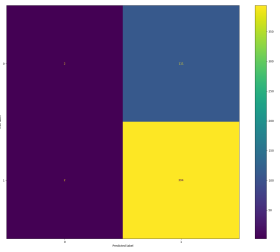
Textula scripts and drag-and-drop Sirius Web diagrams, this feature reduces iteration cycles, minimizes configuration errors, reveals insights in real time, and slashes development time by up to 70%. ML2++ now encompasses the entire model spectrum, featuring classical supervised learners, semi-supervised and unsupervised algorithms, statistical forecasters, and deep learning networks ranging from LSTM and GRU to Transformers. ML2++ cuts development time (e.g., river flow: 14→ 3.5 h) without loss in precision (identical RMSE / ACC). Validated DSL pipelines reduce errors and auto-produce diagnostic plots, while deterministic code generation ensures reproducible results equivalent to handcoded workflows. Our roadmap advances along three converging tracks. Broader deployment will roll out ML2++ across a broad spectrum of multivariate time series data scenarios and validate multiple model instances against both temporal and non-temporal datasets. The models under evaluation will span the full spectrum of techniques, including ARIMA / SARIMA, HoltWinters, Prophet, GRU, LSTM, Transformer, Random Forest, XGBoost, KMeans, DBSCAN, Gaussian Mixture, Logistic Regression, and Support Vector Machines. Each configuration will be benchmarked across classification, regression, clustering, anomaly detection, and multistep forecasting pipelines, ensuring that statistical, classical machine learning, and deep learning approaches are



(a) Correlation heatmap



(b) ROC curve with AUC



(c) Confusion matrix

Fig. 10: Auto-generated diagnostics from **ML2++**: (a) correlation heatmap; (b) ROC curve with AUC; (c) confusion matrix.

compared on equal footing. Systematic user studies will employ structured questionnaires to measure time efficiency, ease of use, learning curve, code quality, maintainability, flexibility, and overall satisfaction. Participants will include ML engineers, data scientists, and domain experts with varying levels of

programming experience; metrics collected will consist of task completion time, error rates, System Usability Scale scores, and qualitative feedback. Iterative refinement will translate insights from these studies into improvements such as context-aware validation messages, enhanced model wizards, clearer tutorials, and richer visual diagnostics. Together, these steps will ensure that ML2++ continues to deliver a comprehensive suite of plots for both ML and time series models while evolving into a transparent, domain-agnostic, and highly user-friendly platform. We will also include head-to-head comparisons with similar approaches on public benchmarks.

Acknowledgments

This work is supported by several projects, including Blockchain.PT (PRR–RE-C05-i01.02: Agendas/Alianças Verdes para a Inovação Empresarial) and the INFLOOD project, funded by FCT through the PRR Investment C05-i08 Ciência Mais Digital (project number 2024.07287.IACDC).

References

[1] D. Zeng, L. Chen, and W. Zhang, "A survey of big data analytics for internet of things: Architectures, enabling technologies, and future directions," *IEEE Internet of Things Journal*, vol. 10, no. 5, pp. 4217–4245, 2023.

[2] L. Zhang and W. Huang, "Challenges in manual time series forecasting: A survey," *International Journal of Forecasting*, vol. 40, no. 1, pp. 1–20, 2024.

[3] Z. Mardani Korani, A. Moin, A. Rodrigues da Silva, and J. C. Ferreira, "Model-driven engineering techniques and tools for machine learning-enabled iot applications: A scoping review," *Sensors*, vol. 23, no. 3, 2023. [Online]. Available: https://www.mdpi.com/1424-8220/23/3/1458

[4] H. Naveed, C. Arora, H. Khalajzadeh, J. Grundy, and O. Haggag, "Model-driven engineering for machine learning components: A systematic literature review," *arXiv preprint arXiv:2311.00284*, 2023.

[5] R. F. Gonçalves and A. Menolli, "Mdd4iot: Model-driven development framework proposal for internet of things," arXiv preprint arXiv:2402.00000, 2024.

[6] A. Moin, M. Challenger, A. Badii, and S. Günnemann, "A model-driven approach to machine learning and software modeling for the iot," *Software and Systems Modeling*, vol. 21, no. 3, pp. 987–1014, 2022.

[7] Z. M. Korani, M. Challenger, A. Moin, J. C. Ferreira, A. R. da Silva, G. V. Jesus, E. L. Alves, and R. Correia, "From ml2 to ml2+: Integrating time series forecasting in model-driven engineering of smart iot applications," in *13th International Conference on Model-Based Software and Systems Engineering (MODELSWARD 2025)*. SCITEPRESS – Science and Technology Publications, 2025, pp. 458–465.

[8] A. C. Marcén, A. Iglesias, R. L. Martí, and C. Cetina, "A systematic literature review of model-driven engineering using machine learning," *Software and Systems Modeling*, vol. 23, no. 3, p. 823–855, 2024.

[9] A. Moin, "Data analytics and machine learning methods, techniques and tool for model-driven engineering of smart iot services," in *2021 IEEE/ACM 43rd International Conference on Software Engineering: Companion Proceedings (ICSE-Companion)*, 2021, pp. 287–292.

[10] M. Abadi, P. Barham, J. Chen, Z. Chen, A. Davis, J. Dean, M. Devin, S. Ghemawat, G. Irving, M. Isard *et al.*, "Tensorflow: Large-scale machine learning on heterogeneous systems," 2015, software available from tensorflow.org.

[11] F. Chollet *et al.*, "Keras," 2015.

[12] M. R. Berthold, N. Cebron, F. Dill, T. R. Gabriel, T. Kötter, T. Meinl, P. Ohl, C. Sieb, K. Thiel, and B. Wiswedel, "Knime: The konstanz information miner," in *Data Analysis, Machine Learning and Applications*. Springer, 2009, pp. 319–326.

[13] I. Mierswa, M. Wurst, R. Klinkenberg, M. Scholz, and T. Euler, "Rapidminer: An open source system for knowledge discovery in large data sets," in *Proceedings of the NIPS ML Open Source Software Workshop*, 2006.

[14] R. L. Grossman and S. Bailey, "Pmml: An open standard for sharing models," *The R Journal*, vol. 5, no. 1, p. 213, 2013.

[15] S. Bai, J. Z. Kolter, and V. Koltun, "Onnx: Open neural network exchange," *arXiv preprint arXiv:1801.06610*, 2018.

[16] Data Mining Group, "Portable format for analytics (pfa)," http://dmg.org/pfa/pfa.html, 2015, accessed: June 2025.

[17] T. P. Minka, J. Winn, J. Guiver, and D. Knowles, "Infer.net: A framework for running bayesian inference in graphical models," *Journal of Machine Learning Research*, vol. 18, pp. 1–5, 2018.

[18] N. Harrand, F. Fleurey, B. Morin, and K. Husa, "Thingml: A language and code generation framework for heterogeneous targets," in *Model-Driven Engineering Languages and Systems*. Springer, 2011, pp. 125–140.

[19] B. Morin, F. Fleurey, N. Bencomo, and J.-M. Jézéquel, "Heads: A methodology for model-based engineering of systems of systems," in *System of Systems Engineering Conference (SoSE)*. IEEE, 2015, pp. 276–281.

[20] F. Fouquet, O. Barais, J. Bourcier, N. Plouzeau, and J.-M. Jezequel, "Leveraging models from design-time to runtime to support dynamic adaptations," in *Computer Software and Applications Conference Workshops (COMPSACW)*. IEEE, 2013, pp. 694–699.

[21] A. Bhattacharjee, Y. Barve, S. Khare, S. Bao, Z. Kang, A. Gokhale, and T. Damiano, "Stratum: A bigdata-as-a-service for lifecycle management of iot analytics applications," in *2019 IEEE International Conference on Big Data (Big Data)*. IEEE, 2019, pp. 1607–1612.

[22] C. Hartsell, N. Mahadevan, S. Ramakrishna, A. Dubey, T. Bapty, T. Johnson, X. Koutsoukos, J. Sztipanovits, and G. Karsai, "Model-based design for cps with learning-enabled components," in *Proceedings of the Workshop on Design Automation for CPS and IoT*, 2019, pp. 1–9.

[23] J. C. Kirchhof, E. Kusmenko, J. Ritz, B. Rumpe, A. Moin, A. Badii, S. Günnemann, and M. Challenger, "Mde for machine learning-enabled software systems: a case study and comparison of montianna & ml-quadrat," in *Proceedings of the 25th International Conference on Model Driven Engineering Languages and Systems: Companion Proceedings*, 2022, pp. 380–387.

[24] T. Hartmann, A. Moawad, F. Fouquet, and Y. Le Traon, "The next evolution of mde: a seamless integration of machine learning into domain modeling," *Software & Systems Modeling*, vol. 18, no. 2, pp. 1285–1304, 2019.

[25] S. Meliá, S. Nasabeh, S. Luján-Mora, and C. Cachero, "Mosiot: Modeling and simulating iot healthcare-monitoring systems for people with disabilities," *International Journal of Environmental Research and Public Health*, vol. 18, no. 12, p. 6357, 2021.

[26] A. Moin, S. Rössler, and S. Günnemann, "Thingml+: Augmenting model-driven software engineering for the internet of things with machine learning," in *Proceedings of MODELS 2018 Workshops, Copenhagen, Denmark, October, 14, 2018*, ser. CEUR Workshop Proceedings, vol. 2245. CEUR-WS.org, 2018, pp. 521–523. [Online]. Available: http://ceur-ws.org/Vol-2245/mde4iot_paper_5.pdf

[27] A. Moin, S. Rössler, M. Sayih, and S. Günnemann, "From things' modeling language (thingml) to things' machine learning (thingml2)," ser. MODELS '20. New York, NY, USA: ACM, 2020. [Online]. Available: https://doi.org/10.1145/3417990.3420057

[28] "Eclipse sirius web," https://eclipse.dev/sirius/sirius-web.html, accessed: 2025-06-21.

[29] "Ml2++ repository (graphical editor / sirius-web viewpoints)," https://github.com/ThimotyS/ML-plusplus/tree/main, accessed: 25 Aug. 2025.

[30] Sistema Nacional de Informação de Recursos Hídricos (SNIRH), "SNIRH: Sistema Nacional de Informação de Recursos Hídricos," https://snirh.apambiente.pt/, 2024, accessed: June 16, 2024.

[31] G. V. Jesus, Z. Mardani Korani, E. Alves, and A. Oliveira, "Deep learning-based river flow forecasting with mlps: Comparative exploratory analysis applied to the tejo and the mondego rivers," *Sensors (Basel)*, vol. 25, no. 7, p. 2154, Mar. 2025.

[32] C. J. Willmott and K. Matsuura, "Advantages of the mean absolute error (mae) over the root mean square error (rmse) in model evaluation," *Climate Research*, vol. 30, no. 1, pp. 79–82, 2005.

[33] "Model evaluation: Accuracy," https://scikit-learn.org/stable/modules/model_evaluation.html#accuracy-score, accessed: 2025-06-21.